

Volume 15

Enterprise API Architecture (EAA)

Version: 1.0

Classification: Enterprise Architecture

Status: Draft

Table of Contents

15.1 Purpose

15.2 API Vision

15.3 API Principles

15.4 API Architecture Overview

15.5 API Classification

15.6 REST API Standards

15.7 API Versioning Strategy

15.8 API Security

15.9 API Gateway Architecture

15.10 Request Processing Pipeline

15.11 API Documentation

15.12 API Lifecycle Management

15.13 Error Handling Standards

15.14 Rate Limiting & Throttling

15.15 Webhooks

15.16 WebSockets & Real-Time Communication

15.17 Future API Evolution

15.18 Volume Summary

15.1 Purpose

The Enterprise API Architecture defines the standards, principles, and governance model for all application programming interfaces exposed by the BiteFixes platform.

The objectives of this architecture are to:

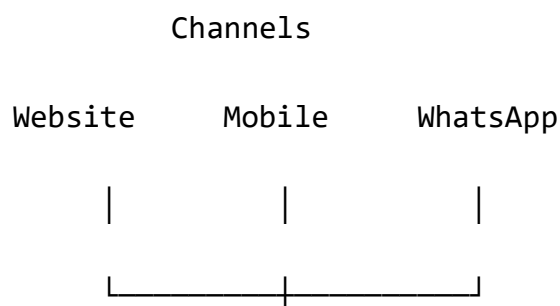
- Ensure consistency across all APIs.
- Support secure communication.
- Enable interoperability between services.
- Simplify integration with external systems.
- Maintain backward compatibility.
- Support long-term scalability.

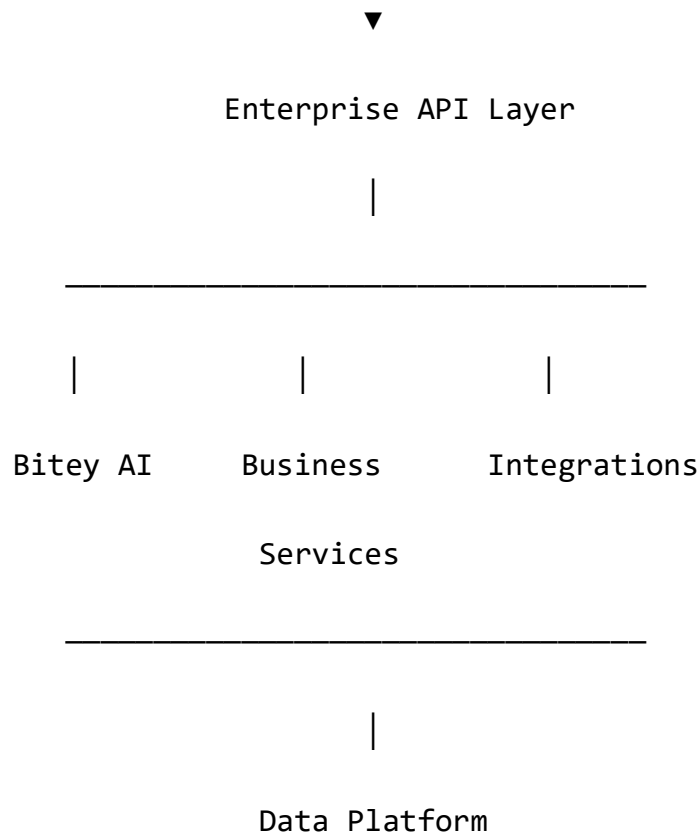
Every public, private, and internal API must comply with this architecture.

15.2 API Vision

APIs are the communication backbone of the BiteFixes platform.

Every business capability is exposed through well-defined service interfaces rather than direct access to internal components.





This architecture ensures that every client consumes the same business capabilities through standardized interfaces.

15.3 API Principles

Principle 1 — API First

Business functionality is designed as reusable APIs before user interfaces are implemented.

Benefits include:

- Consistency
- Reusability
- Faster development
- Easier testing
- Better integrations

Principle 2 — Stateless Communication

Each request contains all information required for processing.

No server-side session state is assumed for API requests.

Authentication and authorization context are supplied with each request.

Principle 3 — Resource-Oriented Design

APIs expose business resources rather than implementation details.

Examples:

Customers

Devices

Tickets

Services

Companies

Knowledge

Conversations

Users

Endpoints represent resources and standard HTTP methods define the operations.

Principle 4 — Consistent Contracts

Every endpoint follows common conventions for:

- Naming
- HTTP methods

- Status codes
- Error responses
- Pagination
- Filtering
- Authentication

Consistency reduces integration complexity.

Principle 5 — Backward Compatibility

Breaking changes are avoided whenever possible.

When incompatible modifications are necessary, a new API version is introduced rather than altering existing behavior.

15.4 API Architecture Overview

The Enterprise API Layer sits between external consumers and internal business services.

External Client

|



API Gateway

|



Authentication

|



Authorization



Business Service



Repository Layer



Database

Each request passes through the complete processing pipeline before reaching business logic.

15.5 API Classification

The platform distinguishes several API categories.

Public APIs

Accessible without authentication.

Examples:

- Health checks
- Public service catalog
- Public company information

Authenticated Customer APIs

Accessible only to authenticated customers.

Examples:

- View tickets
- Create support requests
- Update profile
- View repair status

Technician APIs

Used by repair personnel.

Examples:

- Update diagnostics
- Modify ticket status
- Register repairs
- Add technical notes

Administrative APIs

Reserved for company administrators.

Examples:

- Manage users
- Configure services
- Manage pricing
- Configure business rules

Platform APIs

Reserved for BiteFixes platform administrators.

Examples:

- Tenant management
- Platform monitoring
- Subscription management
- Global configuration

Internal APIs

Never exposed publicly.

Used exclusively for communication between internal services.

Examples:

Identity Service

Notification Service

AI Service

Audit Service

Knowledge Service

Integration APIs

Used by third-party systems.

Examples:

- WhatsApp
- WordPress
- ERP
- CRM
- Payment providers
- Mobile applications

15.6 REST API Standards

Resource Naming

Resources use plural nouns.

Examples:

GET /customers

GET /devices

GET /tickets

GET /services

GET /companies

HTTP Methods

The platform follows standard REST semantics.

Method	Purpose
GET	Retrieve resources
POST	Create resources
PUT	Replace resources
PATCH	Partial update
DELETE	Remove resources

URI Design

Hierarchical relationships are represented through nested resources where appropriate.

Examples:

/customers/{id}

/customers/{id}/devices

/customers/{id}/tickets

/tickets/{id}/comments

/companies/{id}/users

Response Format

All responses use a consistent JSON structure.

Successful responses include:

- Resource data
- Metadata (when applicable)
- Pagination information (for collections)

Error responses follow a standardized schema defined later in this volume.

HTTP Status Codes

Standard status codes are used consistently:

Code	Meaning
200	Success
201	Resource Created
204	No Content
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
409	Conflict
422	Validation Error
429	Too Many Requests
500	Internal Server Error

Idempotency

Operations that modify state should support idempotency where appropriate.

Examples include:

- Payment confirmations
- Webhook processing
- Retryable API operations

Unique request identifiers may be used to detect duplicate submissions.

Chapter Summary

The first half of the Enterprise API Architecture establishes the strategic vision, guiding principles, architectural layers, API categories, and REST standards that govern every interface exposed by the BiteFixes platform. By adopting an API-first approach with consistent contracts, standardized resource design, and clear separation of concerns, the platform ensures maintainability, interoperability, and long-term scalability.

15.7 API Versioning Strategy

Overview

API versioning ensures that the BiteFixes platform can evolve without disrupting existing clients.

Backward compatibility is a fundamental architectural requirement.

Applications already integrated with the platform must continue functioning even as new capabilities are introduced.

Versioning Principles

The versioning strategy follows these principles:

- Backward compatibility by default
- Explicit version identification
- Long-term support for stable versions
- Predictable deprecation process
- Controlled migration

Version Identification

API versions are represented in the URI.

Examples:

GET /api/v1/customers

GET /api/v1/tickets

GET /api/v2/customers

Major versions indicate breaking changes.

Minor improvements remain within the same version.

Version Lifecycle

Development

↓

Beta

↓

Stable

↓

Deprecated

↓

Retired

Clients are informed before a version reaches retirement.

Deprecation Policy

When an endpoint is superseded:

- Documentation is updated.
- Migration guides are published.
- Sunset dates are announced.
- Monitoring identifies remaining consumers.

No version is removed without an appropriate transition period.

15.8 API Security

Overview

Every API request is evaluated against multiple security controls before business logic is executed.

Security is applied consistently across all API categories.

Security Pipeline

Incoming Request

↓

HTTPS Validation

↓

Authentication



Authorization



Input Validation



Business Rules



Audit Logging



Response

Authentication

Supported authentication mechanisms include:

- JWT Bearer Tokens
- API Keys (for integrations)
- OAuth 2.0 (future)
- Single Sign-On (future)

Authorization

Authorization verifies:

- User identity
- Assigned roles
- Permissions
- Company ownership

- Resource ownership

Access is denied unless explicitly authorized.

Transport Security

All API communication uses encrypted channels.

Requirements include:

- HTTPS only
- TLS 1.2 or higher
- Secure certificate management
- HSTS support

Input Protection

Requests are validated against:

- Invalid JSON
- Malformed payloads
- Injection attempts
- Oversized requests
- Invalid encoding
- Unsupported media types

Audit Integration

Every authenticated request generates audit metadata.

Captured information includes:

- User ID
- Company ID
- Endpoint
- Timestamp
- Correlation ID

- Result status

15.9 API Gateway Architecture

Purpose

The API Gateway acts as the unified entry point for all external requests.

It centralizes common infrastructure concerns before requests reach business services.

Gateway Architecture

Clients

|



API Gateway

|

|—— Authentication

|—— Authorization

|—— Rate Limiting

|—— Request Validation

|—— Routing

|—— Logging

|—— Monitoring

|



Business Services

Gateway Responsibilities

The gateway performs:

- Authentication
- Authorization
- Routing
- Rate limiting
- Request logging
- Response compression
- Header normalization
- API version routing
- Metrics collection

Business logic is intentionally excluded from the gateway.

Future Gateway Capabilities

Planned enhancements include:

- Geographic routing
- Intelligent load balancing
- API monetization
- Request caching
- Traffic shaping
- Canary deployments

15.10 Request Processing Pipeline

Overview

Every request follows a deterministic processing pipeline.

This guarantees predictable behavior and simplifies troubleshooting.

Processing Flow

Receive Request

↓

Authentication

↓

Authorization

↓

Schema Validation

↓

Business Validation

↓

Service Execution

↓

Database Transaction

↓

Audit Logging



Response Generation



Response Delivery

Validation Layers

Validation occurs in several stages:

Transport Validation

- HTTPS
- Headers
- Content type

Structural Validation

- JSON schema
- Required fields
- Data types

Business Validation

- Existing customer
- Existing company
- Valid ticket state
- Allowed transitions

Security Validation

- Authentication
- Permissions

- Tenant ownership

Only requests passing all validation stages reach the business services.

Response Standardization

Responses follow a common structure regardless of the endpoint.

Standardized responses simplify client implementations and improve consistency across the platform.

15.11 API Documentation

Documentation Strategy

Every API must be fully documented before production release.

Documentation is treated as part of the product rather than an optional artifact.

Documentation Contents

Each endpoint includes:

- Description
- URI
- HTTP Method
- Parameters
- Request Body
- Response Body
- Status Codes
- Authentication Requirements
- Examples
- Error Scenarios

OpenAPI Standard

The platform adopts the OpenAPI Specification as the primary API contract.

Benefits include:

- Interactive documentation
- SDK generation
- Automated testing
- Contract validation
- Client code generation

Documentation Lifecycle

Design

↓

OpenAPI Definition

↓

Implementation

↓

Testing

↓

Publication

↓

Maintenance

Documentation evolves together with the implementation.

Consumer Documentation

Separate guides are maintained for:

- Internal developers
- Third-party integrators
- Mobile developers
- Enterprise customers
- Platform administrators

This ensures that each audience receives documentation tailored to its use cases.

Chapter Summary

The second section of the Enterprise API Architecture establishes a standardized approach to versioning, security, gateway responsibilities, request processing, and documentation. These practices ensure that every API exposed by BiteFixes remains secure, maintainable, discoverable, and stable as the platform grows and integrates with an increasing number of external systems.

15.12 API Lifecycle Management

Overview

API Lifecycle Management defines the governance processes through which APIs are designed, developed, tested, deployed, maintained, versioned, and retired.

Every API follows a standardized lifecycle to ensure quality, stability, and long-term maintainability.

API Lifecycle

Business Requirement

↓

API Design



Architecture Review



Implementation



Testing



Documentation



Deployment



Monitoring



Maintenance



Deprecation



Retirement

Design Phase

Each API begins with a design review.

The design process defines:

- Business purpose
- Resources
- Operations
- Security requirements
- Data model
- Error handling
- Version strategy
- Performance expectations

Development Phase

Implementation follows enterprise coding standards.

Requirements include:

- Unit tests
- Static analysis
- Code review
- Security validation
- OpenAPI synchronization

Deployment

Only validated APIs are deployed.

Deployment includes:

- Automated testing
- Security checks
- Performance validation
- Documentation publication
- Monitoring activation

Maintenance

Operational APIs are continuously monitored for:

- Performance
- Error rates
- Usage
- Security events
- Consumer adoption

Retirement

Before retiring an API:

- Consumers are notified.
- Migration guides are provided.
- Successor APIs are available.
- Sunset periods are respected.

15.13 Error Handling Standards

Overview

Consistent error handling simplifies client development and operational troubleshooting.

Errors are categorized, standardized, and fully documented.

Error Categories

Validation Errors

Examples:

- Missing fields

- Invalid formats
- Unsupported values

Authentication Errors

Examples:

- Missing token
- Expired token
- Invalid credentials

Authorization Errors

Examples:

- Insufficient permissions
- Tenant mismatch
- Restricted operation

Business Errors

Examples:

- Ticket already closed
- Device already registered
- Invalid workflow transition

Infrastructure Errors

Examples:

- Database unavailable
- External provider failure
- Timeout
- Network interruption

Standard Error Structure

All errors include:

- Error code
- Error message
- Correlation ID
- Timestamp
- Documentation reference

Internal implementation details are never exposed to API consumers.

Error Logging

Every error generates:

- Application log
- Audit event (where applicable)
- Monitoring metrics
- Alert (for critical failures)

15.14 Rate Limiting & Throttling

Objectives

Protect platform availability.

Prevent abuse.

Ensure fair resource allocation.

Mitigate denial-of-service attacks.

Rate Limiting Model

Incoming Requests

↓

Identity Detection

↓

Policy Selection

↓

Request Counter

↓

Within Limit?

↓

YES → Process Request

NO → Reject (429)

Rate Limit Policies

Policies may vary according to consumer type.

Examples:

Consumer	Policy
Anonymous	Very restrictive
Customer	Standard
Technician	Elevated
Company	High
Administrator	Managed
Platform Administrator	Managed
Internal Services	Trusted

Burst Protection

Short traffic bursts are tolerated within configurable thresholds.

Long-term excessive usage triggers throttling.

Monitoring

Metrics include:

- Requests per minute
- Rejected requests
- Peak traffic
- Average throughput
- Consumer behavior

15.15 Webhooks

Overview

Webhooks enable external systems to receive real-time notifications without polling the API.

Webhook Flow

Business Event

↓

Webhook Dispatcher

↓

HTTPS Request



External System



Acknowledgement

Supported Events

Examples:

- Ticket Created
- Ticket Closed
- Customer Registered
- Payment Confirmed
- AI Recommendation Generated

Delivery Guarantees

Webhook delivery includes:

- Retry strategy
- Timeout management
- Signature verification
- Duplicate protection
- Delivery logging

Security

Every webhook request includes:

- Cryptographic signature
- Timestamp
- Event identifier

Consumers verify authenticity before processing.

15.16 WebSockets & Real-Time Communication

Purpose

Real-time communication enables immediate updates without repeated polling.

Supported Scenarios

- Live ticket updates
- Technician notifications
- Dashboard monitoring
- AI conversation streaming
- Administrative alerts

Communication Model

Client

↓

Persistent Connection

↓

WebSocket Gateway

↓

Business Events

↓

Real-Time Updates

Connection Management

The platform manages:

- Authentication
- Authorization
- Heartbeats
- Reconnection
- Session cleanup

Scalability

Future deployments may distribute WebSocket connections across multiple nodes while preserving session consistency.

15.17 Future API Evolution

Vision

The API platform is designed to evolve alongside business requirements without disrupting existing integrations.

Planned Enhancements

Future capabilities include:

- GraphQL endpoints
- gRPC internal communication
- Event-driven APIs
- AI agent APIs

- Partner developer portal
- SDK generation
- API monetization
- Advanced analytics

Long-Term Objectives

Future APIs will support:

- Autonomous AI agents
- Cross-platform orchestration
- Intelligent workflow automation
- Multi-region deployments
- Global enterprise integrations

15.18 Volume Summary

The Enterprise API Architecture establishes the standards, governance model, and operational practices for every API within the BiteFixes ecosystem.

By adopting an API-first strategy, standardized REST conventions, robust security controls, structured lifecycle management, and support for real-time and event-driven communication, the platform creates a stable and extensible integration layer capable of supporting future business growth.

The architecture is intentionally technology-agnostic, allowing the platform to evolve toward GraphQL, gRPC, event streaming, and AI-native interfaces without compromising existing integrations.